10.3 定制与常见问题

> **生成模型**：gpt-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~8k tokens，输出 ~3.1k tokens（估算）

走到这里，其实你已经过了 OpenClaw 最难熬的新手期：环境会配了，模型会接了，渠道能通了，记忆和安全也有基本概念了。接下来真正拉开差距的，是“有没有把它调成适合你自己的样子”。

很多人把 AI 工具用得别扭，不是因为模型不行，而是因为默认行为跟自己的工作方式总打架。你想让它说话短一点，它却爱铺陈；你想让它改代码前先读上下文，它却一上来就动手；你想要一个偏冷静的技术搭子，它偏偏像客服。OpenClaw 给了你一套比较直接的定制方式，关键就是 `AGENTS.md` 和 `SOUL.md` 。

这一节前半段讲怎么把 Agent 调成你顺手的样子，后半段用 FAQ 把新手最常见的问题过一遍。把这节吃透，第 I 部分基本就算收官了。

10.3.1 `AGENTS.md` ： 把项目规则讲清楚

`AGENTS.md` 更像“这个工作区的协作说明书”。它适合写什么？一句话：**凡是跟项目、仓库、流程、约束有关的，都优先放这里。**

比如：

- 代码主要用什么语言。
- 哪些目录能改，哪些目录别碰。
- 改完要跑什么测试。
- 提交信息、分支命名、代码风格有什么约定。
- 文档要用什么语气，注释要不要写中文。

一个很实用的判断方法是：如果换一个项目，这条规则大概率就失效，那它多半应该放 `AGENTS.md` ，而不是 `SOUL.md` 。

比如你在写一本中文技术书，那你完全可以在工作区里写：

```
# 工作区规则
- 默认输出中文
- 不要写太官话，尽量口语化
- 修改文稿时保留元数据头
- 每节结尾加“本节小结”
```

这样做的好处非常明显：你不用每次开新会话都再解释一遍，Agent 一进工作区就先知道这个地方的基本玩法。

10.3.2 `SOUL.md` ：定人格、定口气、定合作习惯

`SOUL.md` 更偏“你希望它像什么样的人”。不是项目规则，而是风格和协作气质。

你可以在这里定义：

- 回答风格是冷静、直接，还是更有陪伴感。
- 默认长答还是短答。
- 遇到不确定时先保守，还是先给可行方案。
- 是不是要主动指出风险。
- 写作时更像工程师、编辑，还是导师。

比如：

```
# 协作风格
- 先说结论，再补背景
- 少用套话，别像客服
- 发现问题直接指出，不要兜圈子
- 技术写作尽量像一个有经验的开发者，不要像宣传文案
```

你会发现， `SOUL.md` 写得好，Agent 的“人味”会稳定很多。它不只是句子变好看，而是合作方式会更顺。比如你不喜欢被反复确认，它就可以更偏执行型；你很在意风险，它就可以更偏谨慎型。

10.3.3 两个文件怎么分工，别写乱了

很多人一开始会把两个文件混着用。最简单的分法其实就一句：

- `AGENTS.md` 管“这份工作怎么做”。
- `SOUL.md` 管“这个助手像什么人”。

举几个例子你就明白了：

- “改代码前先读相关文件” -> 更像工作规则，放 `AGENTS.md` 。
- “说话简洁一点，别摆架子” -> 更像人格风格，放 `SOUL.md` 。
- “这个仓库只能改 `src/` ，别碰 `generated/` ” -> 放 `AGENTS.md` 。
- “指出问题时直接一点，但别阴阳怪气” -> 放 `SOUL.md` 。

分清之后，维护成本会低很多。否则时间一久，这两个文件都会越来越像杂物间。

10.3.4 工作区模板：别每个新项目都从零写

如果你同时在做好几个项目，工作区模板会非常省心。所谓工作区模板，本质上就是把你常用的一套 `AGENTS.md` / `SOUL.md` 预设下来，新项目直接套进去，再做少量修改。

特别适合做模板的场景有：

- 技术写作项目。
- 常规 Web 开发仓库。
- 自动化脚本仓库。
- 团队内部共享的运维项目。

模板的价值不是“更高级”，而是**减少重复配置，降低跑偏概率**。你已经知道自己喜欢什么协作方式，就没必要每个仓库再重新调教一次。

10.3.5 FAQ：新手最容易碰到的问题

下面这组问题，几乎是刚开始日用 OpenClaw 时最常见的一批。

Q1：消息突然不回了，或者看起来像卡住了，先查什么？

先别急着怀疑模型抽风，按这个顺序查最省时间：

1. 跑 `openclaw doctor` 。
2. 看网关进程是不是还活着。
3. 看模型 API Key、OAuth 或网关连接有没有失效。
4. 看渠道自己是不是掉线了，比如 Telegram Bot token、Discord 连接状态。

很多“AI 不回话”的问题，最后都不是 AI 本身，而是链路上某一段断了。

Q2：模型报错，说 unavailable、rate limit、auth failed，这怎么分？

很粗暴但很实用的经验是：

- `auth failed` 往往先查密钥、登录态、profile。
- `rate limit` 先查配额、并发和供应商限速。
- `unavailable` 更像服务端临时波动，等一会儿或者切回退模型。

如果你已经配了模型回退链，很多时候体验会稳很多。别把所有流量都押在一个最贵、最强、也最可能忙的模型上。

Q3：为什么 token 用得这么快？

通常不是一句回复贵，而是**上下文太肥**。常见原因有：

- 会话太长，旧内容一直没压缩。
- 工具结果太多，反复带进上下文。
- 提示文件写得又长又散。
- 你每次都让它输出超长答案。

解决思路也直接：适当 `/compact` ，清理冗余提示，别把所有背景都堆进当前轮，必要时把稳定信息迁到记忆文件里，而不是每次复制粘贴。

Q4：回复怎么越来越慢？

慢，一般就三类原因：网络慢、模型慢、上下文太大。还有一种容易被忽略：工具调用太多。尤其是会读很多文件、跑很多命令的任务，真正耗时的可能根本不在模型，而在工具链。

如果是聊天渠道，块级流式输出通常会比逐字流式更稳，体感也更像真人发消息。

Q5：记忆为什么像没生效？

先问自己三个问题：

- 这件事有没有真的写进 `MEMORY.md` 或 `memory/` ？
- 你记的是不是稳定事实，而不是临时闲聊？
- 向量记忆或检索配置是不是压根没开？

很多人嘴上说“你记住这个”，但实际上那条信息并没有落盘。还有些信息写得太含糊，后来当然也很难搜到。

Q6：我明明配了 `AGENTS.md` / `SOUL.md`，为什么感觉它还是没按我说的来？

通常有三种可能：

- 规则写得太抽象，比如“更聪明一点”“更像人一点”，这种很难稳定执行。
- 规则互相打架，比如既要极度简洁，又要完整解释所有背景。
- 你工作区里有别的上下文文件或系统配置在覆盖它。

写这类文件时，尽量用具体、可执行的话。比如“先给结论，再给两段解释”，就比“结构清晰一些”有效得多。

Q7：沙箱开了以后，有些任务做不成，是不是配置错了？

不一定。很多时候不是错，而是你真的把边界收紧了。安全和自由度本来就会拉扯。正确做法不是一把全关，而是看这件事到底该不该由高风险会话来做。如果确实需要，就单独给可信 agent 开合适权限，而不是把全局门全拆了。

Q8：哪里能求助？

最先看的还是官方文档、项目 README、已有章节和 `doctor` / `security audit` 的输出。再往后，如果是配置层问题，去社区讨论区、Issue、聊天群通常效率更高。提问时尽量带上：

- 你在用什么渠道。
- 报错原文。
- 相关配置片段。
- 复现步骤。

“它坏了，帮我看看”这种求助方式，通常谁都很难帮你定位。

10.3.6 把 OpenClaw 调成“你的”之后，会是什么感觉

当 `AGENTS.md`、`SOUL.md`、记忆和安全策略都慢慢成型之后，你会明显感觉到 OpenClaw 不再只是一个通用大模型外壳，而是一个有你自己工作习惯的系统。它知道这个仓库怎么改，知道你讨厌什么表达方式，知道哪些事情该保守，哪些事情可以更主动一点。

这就是第 I 部分最想帮你走到的位置：不是只会“安装成功”，而是已经能稳定日用。再往后，你可以把它当日常搭子，也可以反过来开始研究它为什么能这样工作。

本节小结

定制 OpenClaw 的关键，不在于写多少配置，而在于分清层次：项目规则放 `AGENTS.md` ，人格与协作风格放 `SOUL.md` ，常用模式沉淀成工作区模板，遇到问题先按链路排查。只要这几件事做顺了，OpenClaw 就会越来越像一个配合默契的长期助手，而不是一台每次都要重新调教的陌生机器。

下一步

第 I 部分到这里就收尾了。接下来怎么读，取决于你更想往哪边走：

- 如果你想搞懂消息是怎么进来、怎么分发的，下一站读第 3 章到第 6 章，重点看网关、协议、会话和路由。
- 如果你最关心 Agent 内部怎么跑、模型怎么切换、上下文怎么组装，继续读第 7 章到第 9 章。
- 如果你想自己扩工具、做自动化、接浏览器和定时任务，第 14 章到第 18 章最对口。
- 如果你对“记忆为什么能检索到”“技能怎么装”“生态怎么扩”更感兴趣，优先看第 20 章到第 22 章；想继续深挖记忆，再看后面的第 28 章专题。
- 如果你准备把 OpenClaw 真正长期在线跑起来，优先看第 23 章到第 30 章，尤其是配置、安全、部署和排障。
- 如果你的目标是最终做出一个类似 OpenClaw 的系统，那就从第二部分开始按顺序读，边读边对照源码和你自己的实验项目。

到这一步，你已经不再只是“会用 OpenClaw”的用户了。接下来，完全可以开始往“看懂 OpenClaw，甚至自己做一个”这条路上走。